Reordenação Neural

Aula 08 — Cross-encoders e o pipeline retrieve & rerank

Projeto Integrador III ▪ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Onde estamos

Aula 07

- Embeddings: LSA no R, modelo neural no Colab
- *Bi-encoder*: consulta e documento viram vetores **separados**
- Híbrido BM25 + denso com RRF

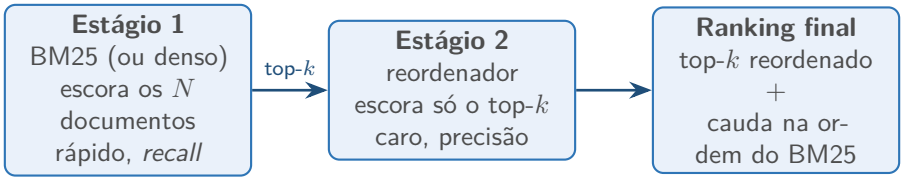
Hoje

- **Reordenação** (*rerank*) do top-k
- *Cross-encoder*: lê consulta e documento **juntos**
- Um reordenador de brinquedo em R; o de verdade no Colab

Ideia

Recuperar rápido os candidatos com o BM25 e depois **reordenar só os primeiros** com um modelo mais caro e mais atento. Tudo nos **nossos 8 documentos**, com a consulta `"modelo de recuperacao"`.

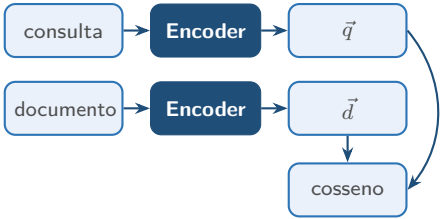
O pipeline *retrieve & rerank*



- O estágio 1 decide **quem entra**: o que ele não traz para o top-k, o estágio 2 nunca vê.
- O estágio 2 decide **a ordem** dentro do top-k: é ali que ficam P@3, RR e nDCG.
- k é um hiperparâmetro novo: quantos candidatos reordenar.

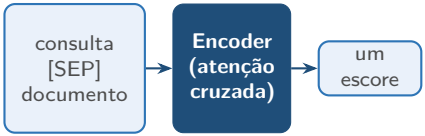
Bi-encoder × *cross-encoder*

Bi-encoder (Aula 07)



- Vetores dos documentos calculados **uma vez**
- Na hora: só o vetor da consulta e cossenos
- Bom para **recuperar**

Cross-encoder (hoje)



- Cada palavra da consulta "olha" cada palavra do documento
- Nada a pré-computar: **um cálculo por par**
- Bom para **reordenar**

Por que não usar o *cross-encoder* em tudo?

- O escore depende do **par** (q, d): só dá para calcular quando a consulta chega.
- Reordenar o *corpus* inteiro $= N$ passagens pelo modelo **por consulta**.
- Reordenar o top-$k = k$ passagens. Com $N = 1$ milhão e $k = 100$: 100 passagens em vez de 1 milhão.

Estágio 1: N escores

Baratos: o índice invertido (Aula 03) só toca documentos com os termos; o BM25 é uma soma.

Estágio 2: k escores

Caros: uma rede com milhões de pesos lê cada par. O custo cresce com k, não com N.

Estágio 1: os 8 documentos e o BM25

```r
docs <- c(
  d1 = "recuperacao de informacao ordena documentos por relevancia",
  d2 = "o modelo de espaco vetorial representa documentos como vetores",
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",
  d5 = "o indice invertido acelera a busca em muitos documentos",
  d6 = "embeddings capturam a semantica de palavras e documentos",
  d7 = "a avaliacao mede a relevancia dos resultados da busca",
  d8 = "ciencia de dados combina estatistica e programacao"
)
tok    <- function(x) unlist(strsplit(tolower(x), "\\s+"))
tokens <- lapply(docs, tok)                # a ORDEM das palavras fica guardada
vocab  <- sort(unique(unlist(tokens)))
tf <- sapply(tokens, function(t) as.integer(table(factor(t, levels = vocab))))
rownames(tf) <- vocab                      # o BM25 so usa contagens
```

Os candidatos do estágio 1

```
q <- "modelo de recuperacao"
r_bm25 <- ranking_bm25(q)
round(r_bm25, 3)

   d3    d1    d2    d4    d8    d6    d5    d7
1.873 1.869 1.687 1.427 0.519 0.492 0.000 0.000

round(metricas(r_bm25, g), 3)       # gabarito da Aula 05, limiar 2

   P_at_k        AP        RR  nDCG_bin nDCG_grad
    0.667     0.833     1.000     0.920     0.951
```

- d3 (1,873) e d1 (1,869) quase empatam no topo; d2 (grau 2) fica em 3º, atrás de d1 (grau 1).
- O BM25 é um **saco de palavras**: d1 tem "recuperacao" e "de", mas em outra frase ("recuperação **de** informação"). A ordem não conta.
- O bi-encoder também não vê isso: consulta e documento viram vetores **antes** de se encontrarem.

Um reordenador que lê consulta e documento juntos

Proximidade ordenada (o nosso brinquedo)

Para cada par de termos distintos da consulta, t_i antes de t_j: se o documento tem t_j **depois** de t_i, a uma distância δ de 1 a 3 palavras, soma $1/\delta$ (a menor distância).

$$\text{prox}(q, d) \;=\; \sum_{i<j} \frac{1}{\min \delta_{ij}} \qquad \delta_{ij} = \text{pos}_d(t_j) - \text{pos}_d(t_i) \in \{1, 2, 3\}$$

- Precisa das **posições** das palavras no documento **e** da ordem da consulta: só dá para calcular com os dois **juntos**, como um cross-encoder.
- É uma regra escrita à mão, não aprendida: serve para ver a **mecânica** do estágio 2.

A proximidade no R

```
proximidade <- function(qt, dt, janela = 3) {
  s <- 0
  for (i in seq_along(qt)) for (j in seq_along(qt))  # todo par da consulta
    if (i < j && qt[i] != qt[j]) {                    # i antes de j, distintos
      pos_i <- which(dt == qt[i])                     # posicoes do termo i em d
      pos_j <- which(dt == qt[j])                     # posicoes do termo j em d
      if (length(pos_i) && length(pos_j)) {
        d <- outer(pos_j, pos_i, "-")                 # todas as pos_j - pos_i
        d <- d[d > 0 & d <= janela]                   # j DEPOIS de i, ate 3
        if (length(d)) s <- s + 1 / min(d)
      }
    }
  s
}
sc <- sapply(tokens, function(dt) proximidade(tok(q), dt))
sc

 d1  d2  d3  d4  d5  d6  d7  d8
0.0 1.0 0.5 0.0 0.0 0.0 0.0 0.0
```

A conta à mão: d2, d3 e d1

Consulta: modelo$_1$ de$_2$ recuperacao$_3$. Pares, na ordem da consulta: (modelo, de), (modelo, recuperacao), (de, recuperacao).

	documento (posições)	pares que contam	prox
d2	o$_1$ **modelo**$_2$ **de**$_3$ espaco$_4$ vetorial$_5$ ...	(modelo, de): $\delta = 1$	$1/1 = 1{,}0$
d3	bm25$_1$ e$_2$ um$_3$ **modelo**$_4$ probabilistico$_5$ **de**$_6$ ranqueamento$_7$ **de**$_8$ texto$_9$	(modelo, de): $\delta \in \{2, 4\}$; só o 2 cabe na janela	$1/2 = 0{,}5$
d1	**recuperacao**$_1$ **de**$_2$ informacao$_3$ ...	(de, recuperacao): $\delta = 1-2 = -1$ (invertida)	0
d4	... **recuperacao**$_5$ moderna$_6$	um termo só: nenhum par	0

d1 tem dois termos da consulta, mas na ordem **oposta** e em outro sentido. O BM25 não distingue; a proximidade, sim.

Reordenar só o top-k

```
reordenar <- function(rk, k, sc) {
  k     <- min(k, length(rk))                    # protecao: k <= numero de docs
  top   <- names(rk)[1:k]                        # os k candidatos do estagio 1
  nova <- top[order(-sc[top], seq_along(top))]   # desempate: ordem do BM25
  c(nova, names(rk)[-(1:k)])                     # cabeca reordenada + cauda
}
reordenar(r_bm25, 2, sc)

[1] "d3" "d1" "d2" "d4" "d8" "d6" "d5" "d7"
```

- Com $k = 2$, os candidatos são d3 (prox 0,5) e d1 (prox 0): a ordem **não muda**.
- d2, o de maior proximidade (1,0), está em 3º — fora do top-2. O reordenador **só mexe no que o estágio 1 trouxe**.
- `order` com dois critérios: o segundo (`seq_along(top)`) desempata pela posição no BM25.

Ranking final: cabeça reordenada + cauda

```
r2 <- reordenar(r_bm25, 2, sc)
r3 <- reordenar(r_bm25, 3, sc)
r3

[1] "d2" "d3" "d1" "d4" "d8" "d6" "d5" "d7"

round(rbind(bm25 = metricas(r_bm25, g), k2 = metricas(r2, g),
            k3 = metricas(r3, g)), 3)

     P_at_k    AP RR nDCG_bin nDCG_grad
bm25  0.667 0.833  1     0.92     0.951
k2    0.667 0.833  1     0.92     0.951
k3    0.667 1.000  1     1.00     0.982
```

- $k = 3$: d2 (1,0) sobe para 1º, d3 (0,5) fica em 2º, d1 (0) cai para 3º; a cauda não se move.
- Relevantes nas posições 1 e 2: AP de 0,833 para **1**; nDCG graduado de 0,951 para **0,982**.
- $k = 4$, 8 ou 20 (a proteção corta para 8): o mesmo ranking (os outros têm prox 0; o empate mantém a ordem do BM25).

Os limites do brinquedo

```
q2 <- "busca em documentos"
round(ranking_bm25(q2), 3)

   d5    d7    d1    d6    d2    d3    d4    d8
3.583 1.219 0.731 0.693 0.659 0.000 0.000 0.000

round(sapply(tokens, function(dt) proximidade(tok(q2), dt)), 3)

   d1    d2    d3    d4    d5    d6    d7    d8
0.000 0.000 0.000 0.000 1.833 0.000 0.000 0.000

sapply(tokens, function(dt) proximidade(tok("documentos busca"), dt))

d1 d2 d3 d4 d5 d6 d7 d8
 0  0  0  0  0  0  0  0
```

- "busca em documentos": só d5 pontua ("busca em muitos documentos": $1 + \frac{1}{2} + \frac{1}{3}$), mas já era o 1º.
- "documentos busca": **zero em todos** — a regra depende da ordem literal.

Os reordenadores de verdade

- **MonoBERT** (Nogueira & Cho, 2019): BERT lê "consulta [SEP] documento" e classifica o par como relevante ou não; a probabilidade é o escore.
- **MonoT5** (Nogueira et al., 2020): um modelo *seq2seq* que responde "true"/"false"; o escore é a probabilidade de "true".
- **Cross-encoders MiniLM**: versões menores e mais rápidas (biblioteca *sentence-transformers*).

O que usaremos no projeto

`cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`: multilíngue, treinado no **mMARCO** (o MS MARCO traduzido, inclusive para o português) — **milhões** de pares (pergunta, trecho) julgados. Mesma ideia do brinquedo: ler os dois **juntos**; o resto foi aprendido.

Latência × qualidade: escolher k

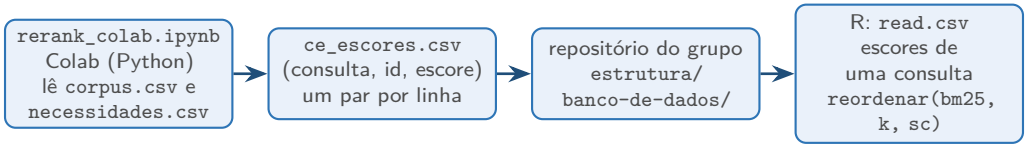
- k maior: mais chance de o relevante estar entre os candidatos (aqui, $k = 2$ não bastou) — e mais pares para o modelo.
- k menor: mais rápido, mas o estágio 1 limita o ganho.
- $k = 0$: não reordenar. Tem de ser um candidato honesto: o rerank pode **piorar**.

- Em produção: modelos menores (destilação), GPU, cache de consultas frequentes.
- No projeto: o *corpus* é pequeno e os escores vêm prontos de um CSV; o custo é uma vez só, no Colab.

Como k_1, b, β e o k do LSA

k é um hiperparâmetro: escolher no **dev**, relatar no **teste** uma vez. Medir qualidade *e* tempo de resposta.

No projeto: Colab (Python) → CSV → R



- O notebook abre direto do GitHub do curso (`ferramentas/rerank_colab.ipynb`) e escora **todos** os pares (consulta, documento).
- No R, os escores do CSV entram no lugar de `sc`: a mesma `reordenar`, só no top-k do BM25.
- Conferir: todos os `id` do `corpus.csv` e todas as consultas aparecem? Uma linha por par?
- Depois: MAP com e sem rerank para vários k; a decisão vai para o `config.R` (`rerank_k`).

Para casa

Tarefa (Parte D)

1. Gerar o `ce_escores.csv` no Colab, subir para `estrutura/banco-de-dados/`, ler no R e conferir.
2. Numa consulta, reordenar o top-k do BM25 e comparar o top-5 antes e depois.
3. MAP do BM25 $\times$ BM25 + rerank, $k \in \{5, 10, 20\}$: escolher no **dev**, relatar no **teste**.
4. Registrar `rerank_k` no `config.R` (0 = não reordenar) e na ficha.

Próxima aula

- *Learning to Rank*

Leitura

- Lin, Nogueira & Yates (2021), cap. 3
- Nogueira & Cho (2019), *Passage Re-ranking with BERT*

Obrigado!

Dúvidas? Vamos ao R.